

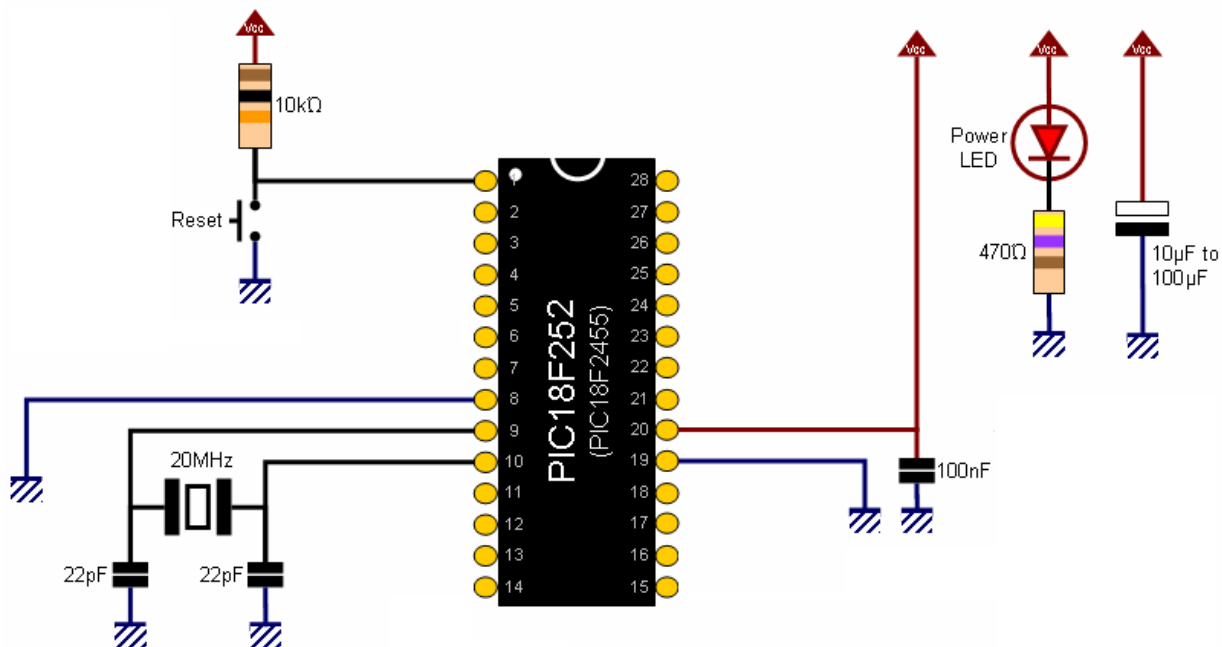
# TALLER V - MICROCONTROLADORES

## Escuela de Física - Facultad de Ciencias

Universidad Nacional de Colombia - Sede Medellín - 2016

PROFESOR: NERIO ANDRÉS MONTOYA - CARLOS TRUJILLO

[namontoy@unal.edu.co](mailto:namontoy@unal.edu.co) - [catrujila@unal.edu.co](mailto:catrujila@unal.edu.co)



## Examen Final - Taller V

### Objetivos

- Evaluar la capacidad de cada estudiante para programar un microcontrolador.
- Utilizar un microcontrolador diferente a los utilizados en el desarrollo del curso.

- 
- Verificar que se tienen los conocimientos mínimos para trabajar con diferentes microcontroladores.
  - Aprender un método para cambiar el comportamiento del microcontrolador sin tener la necesidad de reprogramar el micro.

## Introducción

El siguiente examen es un poco “particular” en el sentido que es una actividad guiada, con la que se desea que el estudiante llegue más lejos de los conocimientos previos adquiridos en el curso y con la que puede profundizar en el manejo de los microcontroladores, mientras es evaluado.

Para desarrollar esta actividad, cada estudiante recibirá un conjunto de microcontrolador y cristal, los cuales en general son diferentes a los utilizados de forma tradicional en el curso, para generar la necesidad en el estudiante de reevaluar todo el conocimiento adquirido durante el curso y someterlo a una realidad del día a día en la vida fuera de la U, donde la necesidad de utilizar otros dispositivos diferentes a los conocidos es lo más común.

Además del microcontrolador y el cristal, al estudiante se le entregará un programa básico, desde el cual se debe partir para desarrollar toda la actividad evaluativa.

Todos los pasos descritos a continuación se hacen basados en el ambiente de desarrollo **MPLAB-X v3.40** (compilador XC8 v1.38), el sistema operativo **Linux OpenSUSE 13.2**, el programador **Pickit 3** y la interface de comunicación serial **minicom** (terminal).

A cada estudiante se le entregará un microcontrolador que puede ser:

- PIC16F877A
- PIC16F873A
- PIC16F88
- PIC16F883
- PIC16F886

- 
- PIC12F683 (caso especial).

Y un cristal que puede ser de 4MHz, 10MHz o 16MHz.

De tal forma que cada estudiante puede tener una solución similar pero con características diferentes debido a la combinación micro/cristal.

El objetivo final del programa que se desea desarrollar es que al micro se le puedan enviar comandos desde una interface serial y que el micro responda adecuadamente. Este tipo de comandos son elementos más complejos que simples envíos de teclas como 'j' o 'm', pasando a ser cadenas de caracteres con las cuales se puede tener comandos que reciben parámetros, los cuales pueden modificar el funcionamiento del micro sin la necesidad de reprogramarlo.

Para el caso en cuestión y debido a la necesidad de cumplir con algunas restricciones necesarias para el funcionamiento adecuado del equipo, todos los comandos que se van a ejecutar serán del tipo: "comando número número", por ejemplo:

```
setADC 0 10<enter>
```

Nota: El símbolo <enter> significa que se presiona la tecla Enter.

Observe que en el ejemplo, los números van separados por un (1) símbolo espacio. Esta convención debe ser respetada siempre. Dado el caso que se tenga un comando que no requiera números, entonces se podrá introducir dos valores cualquiera, ya que el programa deberá ser capaz de simplemente no hacer nada con ellos. Por defecto, estos valores (si no se necesitan) pueden ser el valor cero (0). Imagine por ejemplo que tiene un comando llamado setPin, el cual requiere solo un parámetro con valor cero (0) o uno (1), este comando se puede escribir como:

```
setPin 1 0<enter>
```

Y se debe obtener como resultado que el pin deseado se enciende.

---

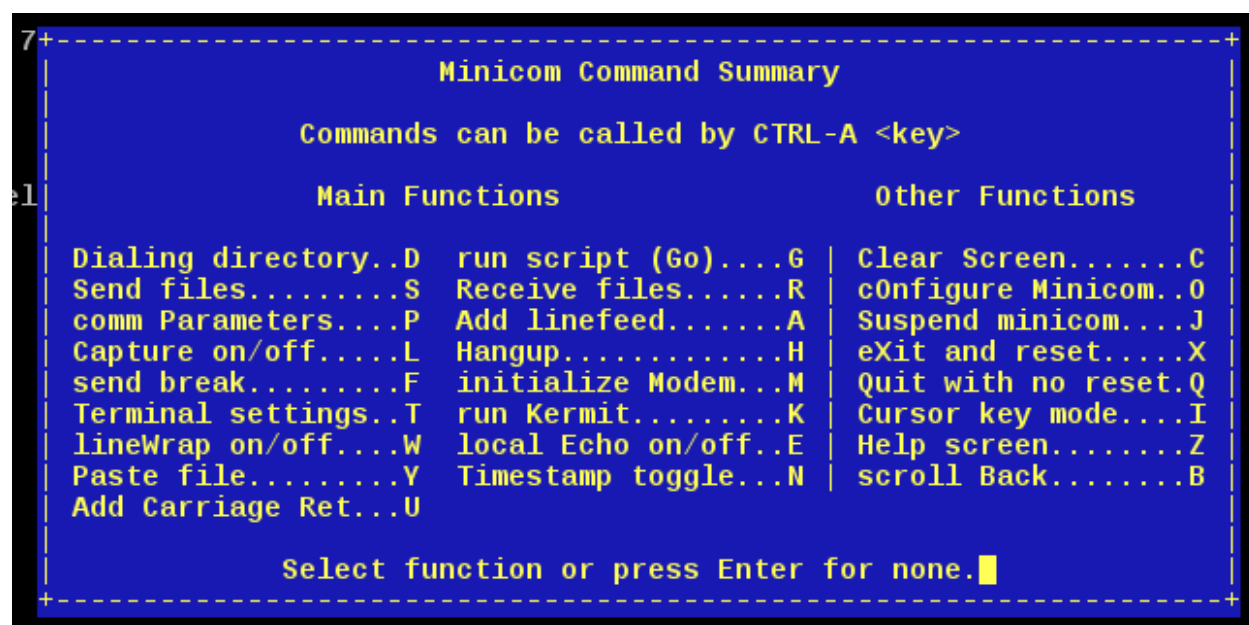
También puede pensarse que el primer parámetro sea el número del pin y el segundo sea el estado (1 -> encendido, 2 -> apagado), de esta forma el comando

setPin 3 1<enter>

Indicaría que el pin "3" debe ser puesto en 1 lógico, o sea encendido.

Imagine un comando que sea "setLedState 250 0<enter>". Podría significar que le asignamos un periodo de 250ms a un timer que nos controla el led de estado.

NOTA: Para el caso de minicom, lo debe configurar para que cada vez que se presione la tecla enter, el minicom envíe los caracteres LF+CR, para esto debe entrar a minicom y cuando este ya recibiendo datos del micro abrir todas las opciones (teclas CTRL+A y luego la opción Z). Se debe desplegar un menú con muchas opciones, entre las cuales está U, que es para activar el envío del CR. La imagen muestra las opciones del minicom, en la esquina inferior izquierda se ve el comando U.



---

## Procedimiento

A continuación se describen los pasos a seguir para desarrollar los puntos del examen. Si usted sigue con juicio estos puntos, con seguridad podrá llevar a cabo y terminar la tarea felizmente.

A no ser que se indique otra cosa, todos los pasos descritos relacionados con programación, deben ser desarrollados en el código de la clase main:

1. Instale su microcontrolador en la board, conectando adecuadamente los pines de alimentación (Vdd y GND), el cristal y sus capacitores, el MCLR, y los pines asociados a la programación. También tenga en cuenta los pines para la comunicación serial con su respectivo cable USB-Serial.
2. Abra el proyecto que se le ha entregado y genere los bits de configuración adecuados teniendo en cuenta el cristal que se le entregó, además de tener en cuenta todas las otras configuraciones, como desactivar el Watch Dog Timer, configurar el Power On Timer, y el Brown Out.
3. Verifique que el programa compile. No debería tener ningún warning "grave" u otros inconvenientes. Para los estudiantes que tienen el micro PIC16F88, les debe salir un error debido a que el registro TRISC no existe. Además, deben salir unos warnings que advierten sobre funciones que no se están utilizando, lo cual es normal porque aun no se han implementado en el código que se les entregó.
4. Organice el código correspondiente en el puerto serial para que el equipo Transmita/Reciba datos a 9600bps en configuración 8N1.
5. Organice el código correspondiente al módulo del led de estado para que este tenga un periodo de 500ms (o sea 250ms encendido y 250ms apagado).
6. Verifique que las librerías <stdbool.h>, <string.h>, <stdio.h> están incluidas en el main.

- 
7. Cree las variables que se presentan en la imagen de la siguiente página. Por favor copie los nombres, tipos y tamaños iguales.
  8. Observe que su led de estado debe estar parpadeando y el micro ya debe estar en la capacidad de enviar el mensaje de saludo. Dentro del while infinito agregue dos nuevos bloques **if()** independientes, uno para cuando la variable receptionChar es diferente del carácter NULL y el otro para cuando la variable receptionFinish es

```
#define BUFFER_SIZE 26
#define BUFFER_SIZE_TRANSMISION 32
#define COMMAND_SIZE 14
#define MIN_CMD_SIZE 6

/*Variables propias del main*/
int firstParameter;
int secondParameter;
char contadorDatosRx;
char *ptrReception;
bool receptionFinish;
char bufferReception[BUFFER_SIZE];
char bufferTrasmision[BUFFER_SIZE_TRANSMISION];
char command[COMMAND_SIZE];

const char helpMenu[] = {
    "\n\r 1) helpMenu - Print this help \n\r 2) setPin #pin newState - Set #pin to newState\n\r "
};
```

verdadera.

9. Dentro de la función main, antes de llamar a las funciones de configuración, inicialice la variable "receptionFinish" como false, la variable contadorDatosRx en cero y el puntero ptrReception apuntando al inicio del arreglo bufferReception.
10. Justo después del mensaje de bienvenida, agregue una línea que imprima el arreglo helpMenu.
11. Dentro del if() para la variable receptionChar, agregue una línea que retorne a la terminal el valor recibido por el micro a través de la recepción serial.

---

12. Justo después de agregar el envío del carácter, agregue un bloque switch-case para la variable receptionChar en el cual se analizaran 4 casos:

- Si receptionChar es igual a '\b'
- Si receptionChar es igual a '\r'
- Si receptionChar es igual a '\n'
- Si receptionChar es cualquier otro caso....

**IMPORTANTE:** Los casos \r y \n solo se dan juntos (porque son el <enter>), por lo cual se deja el caso de \r vacío (y sin el break), de forma que cuando llegue \r se ejecuta el caso \n.

13. A continuación se presentan 2 imagenes, las cuales corresponden a 2 de los 3 casos del switch-case que se deben programar. Agregue el código de la imagen en donde sea correcto (se deduce por el código que presenta la imagen y la funcionalidad que

```
{
*ptrReception = '\0'; // Agregamos el caracter null para que tenga el formato de un String normal
if (contadorDatosRx > MIN_CMD_SIZE) {
    parseCommand(&bufferReception[0], &command[0], &firstParameter, &secondParameter);
    receptionFinish = true;
}

// Como se recibio un comando, bueno o malo, debemos reiniciar el ptr para que reciba otro
ptrReception = &bufferReception[0];

// Devolvemos el contador a cero, para de nuevo contar los caracteres recibidos
contadorDatosRx = 0;

// Enviamos un LF+CR que es la representacion del <enter>
sendString("\n\r");
}

break;
```

se desea obtener).

Y el otro código es

---

```

{
    if (ptrReception > &bufferReception[0]) {
        // Eliminamos del arreglo los dos elementos que se recibieron (el dato malo)
        *ptrReception-- = '\0';
        *ptrReception = '\0';
        // Borramos en la consola
        sendString(" \b");

        if (ptrReception == &bufferReception[0]) {
            ptrReception = &bufferReception[0];
            contadorDatosRx = 0;
        }
    }
}
break;

```

---

14. El último caso debe ser programado por usted. Tenga en cuenta:

- Los datos que se reciben deben ser contados allí (es lo primero).
- Agregar el dato que se recibe al buffer de recepción utilizando para esto un puntero y verificando que el tamaño del buffer siempre sea menor al número de datos recibidos (es el segundo paso).
- Finalmente, si sucede que el contador de elementos recibidos es mas grande que el tamaño del buffer, entonces debe disminuir en 1 el contador de datos, almacenar el dato en la posición a la que apunta el puntero sin aumentar el puntero para una siguiente posición y enviar a la consola 2 veces el símbolo del backspace y luego el símbolo '\$', para indicarle al usuario que llego al limite del arreglo y que no es posible escribir mas.

15. Finalmente, después de los casos del switch-case, justo después de terminar este bloque, debe asignar el caracter NULL a la variable receptionChar, para que solo se vuelva a ejecutar esta tarea cuando se reciba un nuevo dato.

16. La siguiente tarea, que es la correspondiente a receptionFinish se muestra en la imagen.



---

```
if (receptionFinish == true) {
    if (!strcmp(command, "helpMenu")) {
        sendString(helpMenu);
    }
    if (!strcmp(command, "setPin")) {
        sendString(command);
        sendString("\n\r");
        setPinState(&firstParameter, &secondParameter);
    }
    receptionFinish = false;
}
```

17. Observe que allí se encuentra descrito el funcionamiento de dos comandos, uno llamado helpMenu y el otro llamado setPin. De esta misma forma deben ser los otros comandos. Observe además que la función setPinState recibe punteros a variables y no a las variables propiamente dichas, así deben ser todas las funciones que se le pide que sean creadas.

18. Las funciones (y sus comandos en consola) que debe crear son:

- Una función que le cambie el periodo en 'ms' el parpadeo del led de estado. Debe ser posible tener varios estados.
- Una función que permita controlar la frecuencia y el duty de una señal PWM. La frecuencia debe poder variar entre 3KHz y 10KHz (pueden ser valores enteros: 3, 4, 5...), además, a el duty se le debe poder asignar un valor entre 10% a 90%, también con pasos enteros de 10, o sea 10, 20, 30, etc.
- Una función que presente una conversión ADC sencilla en el PIN AN0. Hacer la conversión en modo sleep. Utilice una tabla de lookup para los datos

---

obtenidos, de forma que se entreguen los valores en mV y los rangos del conversor.

- Una función que presente una conversión ADC sencilla en el PIN AN0 con los voltajes de referencia en  $V_{ref+} = 4V_{dc}$  y  $V_{ref-} = 0.8V_{dc}$ . Hacer la conversión en modo sleep. Utilice una tabla de lookup para los datos obtenidos, de forma que se entreguen los valores en mV y los rangos del conversor.
- Una función que me entregue el valor de una captura de frecuencia, de una señal que puede variar entre los 5KHz a los 10KHz. **NO** utilice una tabla de lookup para la captura de frecuencia. Imprima el valor obtenido y la información necesaria para interpretar el dato, o sea, cada cuanto se está incrementando el Timer y una explicación de cómo se debe hacer la multiplicación (o sea, imprima un String con esta explicación)
- Además de esto, por favor identifique 10 claras diferencias entre el hardware del micro que se le asignó para el examen y el micro que se utiliza en el curso.

### Observaciones:

- El led de estado debe estar presente.

***Fecha de entrega:*** *Hasta el Martes 15 de Noviembre de 2016, antes de iniciar la clase.*

*(Curso Carlos Trujillo) Hasta el Miércoles 16 de Noviembre de 2016, antes de iniciar la clase.*